作业报告：Clang 编译器逻辑阶段剖析

姓名：陈嘉豪

学号：2023303961

一、安装

安装方式：安装LLVM 官方安装包，安装时勾选**Add LLVM to system PATH**，直接在 CMD/PowerShell 中使用 clang/lldb 命令。（我安装的是21.1.6 version）

LLVM 提供了详尽的 HTML 格式文档，这些文档可在 可下载原始码并在线访问。

日期	版本	下载	发行说明	文献资料
现状	Git	来源：Git	发行说明	纪录片
2026年2月24日	22.1.0	下载	LLVM clang LLD clang-extra libc++ Polly Flang	LLVM clang LLD clang-extra libc++ Polly Flang
2025年11月18日	21.1.6	下载	LLVM clang LLD clang-extra libc++ Polly Flang	LLVM clang LLD clang-extra libc++ Polly Flang
2025年8月26日	21.1.0	下载	LLVM clang LLD clang-extra libc++ Polly Flang	LLVM clang LLD clang-extra libc++ Polly Flang
2025年3月4日	20.1.0	下载	LLVM clang LLD clang-extra libc++ Polly Flang	LLVM clang LLD clang-extra libc++ Polly Flang
2024年10月1日	19.1.1	下载	发行说明	
2024年9月17日	19.1.0	下载	LLVM clang LLD clang-extra libc++ Polly Flang	LLVM clang LLD clang-extra libc++ Polly Flang

进入github：Release LLVM 21.1.6 · llvm/llvm-project · GitHub（x64就行）

LLVM 21.1.6 Release

- Linux x86_64 (signature)
- Linux Arm64 (signature)
- Linux Armv7-a (signature)
- macOS Apple Silicon (ARM64) (signature)
- Windows x64 (64-bit): installer (signature), archive (signature)
- Windows x86 (32-bit): installer (signature)
- Windows on Arm (ARM64): installer (signature), archive (signature)

1. 打开命令行工具

- Windows：按下 `Win + R` → 输入 `cmd` → 回车（打开 “命令提示符”，黑色窗口）；
- （可选）也可以用 PowerShell，操作完全一致。

2. 验证 Clang 是否安装成功（关键！）

```
clang --version
```

预期结果：

如果安装成功，会显示类似这样的版本信息（版本号可能不同）：

```
C:\Users\CJH>clang --version
clang version 21.1.6
Target: x86_64-pc-windows-msvc
Thread model: posix
InstalledDir: D:\Program Files\LLVM\bin

C:\Users\CJH>
```

二、运行

1.初始化

```
cd /d %USERPROFILE%\Desktop
notepad test.c
```

```c
#include <stdio.h>

int main() {
    int a[10];
    int index = 2;
    int x = 4;
    int y = 2;
    a[index] = x + y;
    printf("a[%d] = %d\n", index, a[index]);
    return 0;
}
```

2. 词法分析阶段 (Scanning) —— 对应你的 CMD 输出

```
clang -fsyntax-only -Xclang -dump-tokens test.c > tokens.txt
```

课件定义：将源程序字符流切割成单词（Token）。 **实验现象定位**： 在你的输出中，可以看到编译器精准地识别了每一个符号：

- **关键字**：`int`，`return`
- **标识符**：`main`，`a`，`index`，`x`，`y`
- **常数**：`numeric_constant '10'`，`'2'`，`'4'`
- **运算符**：`equal '='`，`plus '+'`
- **界符**：`l_bracket '['`，`semi ';'`，`l_brace '{'`

> **关键发现**：在输出的第一部分（那些路径极长的行），你可以看到编译器正在扫描 `stdio.h`。这说明**预处理（Preprocessing）**已经完成，Clang 正在把头文件的内容也一起进行词法分析。

---

3. 语法与语义分析阶段 (Parsing & Semantic) —— 对应 `ast.txt`

```
clang -fsyntax-only -Xclang -ast-dump test.c > ast.txt
```

课件定义：生成语法树并检查语义（如类型是否匹配）。 **实验现象定位**：

在 `ast.txt` 的末尾（line 1083 左右），我们找到了处理 `a[index] = x + y` 的核心逻辑：

- **BinaryOperator**：对应 `index` 位置的 `=`。
- **ImplicitCastExpr**：出现了多次。比如 `<LValueToRValue>`，这证明语义分析程序正在工作——它知道 `x` 和 `y` 不是作为变量名存在，而是要取它们内存里的**值**。
- **类型识别**：每个节点后面都跟了 `'int'`，说明编译器已经完成了类型审查。

---

4. 中间代码生成 (Intermediate Code) —— 对应 `test_unopt.ll`

```
clang -S -emit-llvm -O0 test.c -o test_unopt.ll
```

课件定义：生成三地址码。 **实验现象定位**： 在 `test_unopt.ll` 的 `main` 函数部分（line 92 之后），虽然 LLVM 的格式比课件里的伪代码复杂，但逻辑完全一致：

- **课件的** `t1 = x + y`：对应指令 `%10 = add nsw i32 %8, %9`。
- **课件的** `t2 = index \* 4`：对应指令 `getelementptr inbounds [10 x i32], ptr %2, i64 0, i64 %5`。LLVM 将索引乘以类型大小（4字节）的过程封装在了这个指令里。
- **课件的** `\*t3 = t1`：对应指令 `store i32 %10, ptr %11`。

---

5. 代码优化阶段 (Optimization) —— 对应 `test_opt.ll`

```
clang -S -emit-llvm -O2 test.c -o test_opt.ll
```

课件定义：精简中间代码，如常量合并。 **实验现象定位**： 对比 `test_unopt.ll` 和 `test_opt.ll`：

- **未优化版**：有大量的 `alloca`（分配空间）和 `load/store`（内存读写）。
- **优化版 (`test_opt.ll`)**：你会发现原本的 `index`, `x`, `y` 变量消失了。
- **关键差异**：优化器发现 `x=4, y=2`，所以它直接计算出了结果。在调用 `printf` 时，它直接传递了常量 `2`（索引）和 `6`（结果）。

---

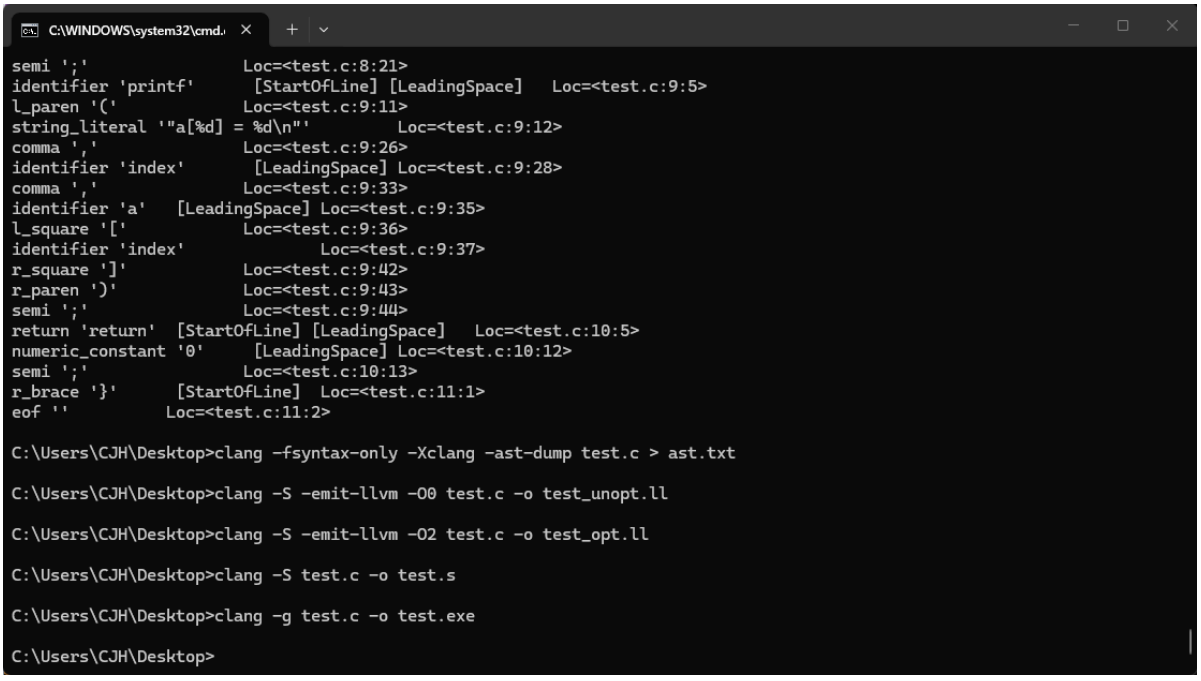
6. 目标代码生成 (Backend) —— 对应 `test.s`

```
clang -S test.c -o test.s
```

课件定义：翻译成机器能执行的汇编。 **实验现象定位**： 在 `test.s` 中，你会看到 Windows 风格的 x86-64 汇编：

- **常量载入**：`movl $2, %r8d` （将 2 放入寄存器，对应 index）。
- **计算结果**：`movl $6, 104(%rsp)` （直接把计算好的 6 存入栈，对应 a[index]）。
- **函数调用**：`callq printf`。

逻辑阶段	输入	你的输出产物	体现的编译器组件
词法分析	字符流	CMD 中的 Token 序列	扫描器 (Scanner)
语法分析	Token 流	`ast.txt` 的层级结构	分析器 (Parser)
语义分析	语法树	`ast.txt` 中的类型标注	语义分析器
中间代码生成	语法树	`test_unopt.ll` (IR)	前端代码生成器
代码优化	IR	`test_opt.ll` (精简 IR)	优化器 (Optimizer)
目标代码生成	优化后的 IR	`test.s` (汇编)	后端 (Backend)



名称	修改日期	类型	大小
ast	14/3/2026 下午10:43	文本文档	752 KB
test	14/3/2026 下午10:18	C 源文件	1 KB
test	14/3/2026 下午10:43	应用程序	630 KB
test.ilk	14/3/2026 下午10:43	Incremental Link...	3,827 KB
test.pdb	14/3/2026 下午10:43	Program Debug...	6,692 KB
test.s	14/3/2026 下午10:43	Assembler Source	11 KB
test_opt.ll	14/3/2026 下午10:43	LL 文件	5 KB
test_unopt.ll	14/3/2026 下午10:43	LL 文件	10 KB
tokens	14/3/2026 下午10:42	文本文档	0 KB

理论上最终有这些文件（pdb）

三、调试

```
clang -g test.c -o test.exe
```

```
lldb test.exe
```

```
C:\Users\CJH\Desktop>lldb test.exe
(lldb) target create "test.exe"
Current executable set to 'C:\Users\CJH\Desktop\test.exe' (x86_64).
(lldb)
```

成功进入调试界面后，我们可以通过以下指令观察编译器的“成果”：

- **打断点 (Breakpoint)**： `b main` *这是在告诉调试器：找到符号表里* `main` *函数对应的物理地址，并在这里拦截。*
- **运行 (Run)**： `run` *程序会停在* `int a[10];` *这一行。*
- **查看变量内存 (Variables)**： 输入 `frame variable` *你会看到* `a`, `index`, `x`, `y` *的当前值。这体现了编译器在**语义分析阶段**建立的**符号表**：它知道每个名字对应内存里的哪个位置，以及它是什么类型。*
- **单步执行 (Step)**： 输入 `next` (或者缩写 `n`) *每按一次回车，程序执行一行。你可以观察* `a[index] = x + y` *执行完后，内存里对应的位置是不是变成了* `6`*。*

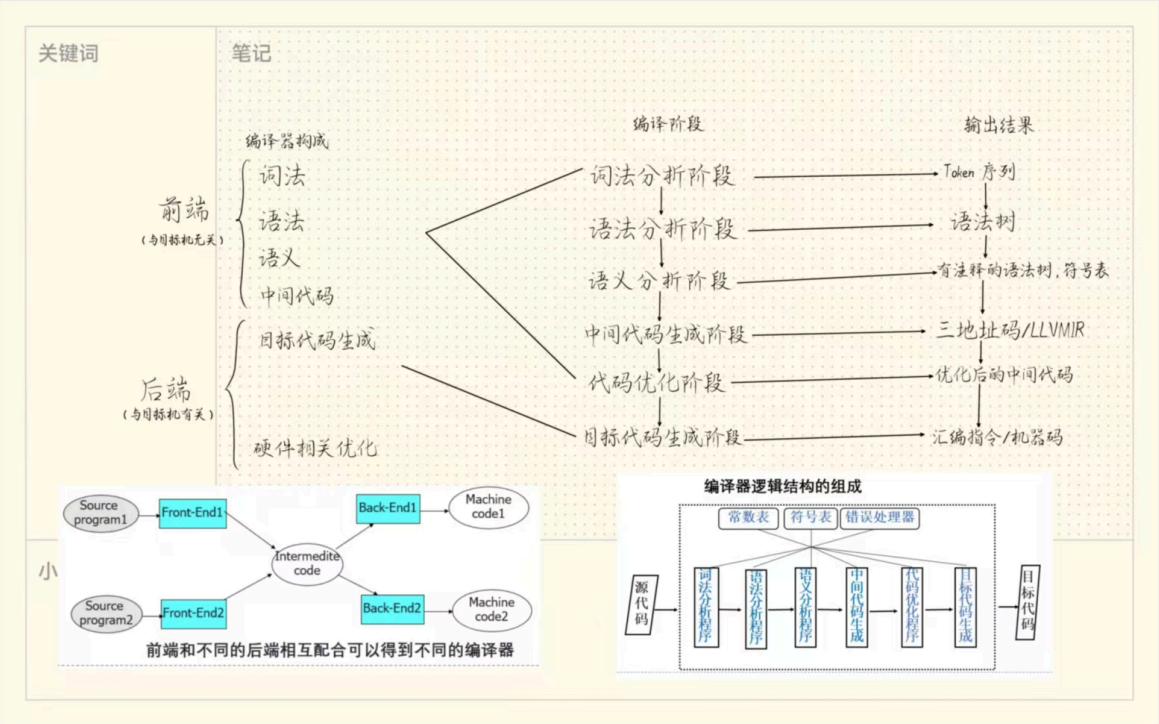
- C:\WINDOWS\system32\cmd.

```
Microsoft Windows [版本 10.0.26200.7840]
(c) Microsoft Corporation。保留所有权利。

C:\Users\CJH>cd /d %USERPROFILE%\Desktop

C:\Users\CJH\Desktop>lldb test.exe
(lldb) target create "test.exe"
Current executable set to 'C:\Users\CJH\Desktop\test.exe' (x86_64).
(lldb) b main
Breakpoint 1: where = test.exe`main at test.c:3, address = 0x0000000140007170
(lldb) run
Process 56228 launched: 'C:\Users\CJH\Desktop\test.exe' (x86_64)
Process 56228 stopped
* thread #1, stop reason = breakpoint 1.1
    frame #0: 0x00007ff7d4707170 test.exe`main at test.c:3
   1    #include <stdio.h>
   2
-> 3    int main() {
   4        int a[10];
   5        int index = 2;
   6        int x = 4;
   7        int y = 2;
(lldb) n
Process 56228 exited with status = -1073741510 (0xc000013a)
(lldb) v x
error: Command requires a process which is currently stopped.
(lldb) v y
error: Command requires a process which is currently stopped.
(lldb) v index
error: Command requires a process which is currently stopped.
```

- C:\WINDOWS\system32\cmd.

```
Process 52112 stopped
* thread #1, stop reason = step over
    frame #0: 0x00007ff7d470717c test.exe`main at test.c:5
   2
   3    int main() {
   4        int a[10];
-> 5        int index = 2;
   6        int x = 4;
   7        int y = 2;
   8        a[index] = x + y;
(lldb) n
Process 52112 stopped
* thread #1, stop reason = step over
    frame #0: 0x00007ff7d4707184 test.exe`main at test.c:6
   3    int main() {
   4        int a[10];
   5        int index = 2;
-> 6        int x = 4;
   7        int y = 2;
   8        a[index] = x + y;
   9        printf("a[%d] = %d\n", index, a[index]);
(lldb) v x
(int) x = 2147154879
(lldb) v y
(int) y = 276826112
(lldb) v index
(int) index = 2
(lldb) v a[2]
(int) a[2] = -730814939
(lldb)
```

四、总结



来自华为笔记

课件阶段	对应你的文件	关键指令/内容
词法分析	CMD 输出	`numeric_constant '4'`
语法/语义	`ast.txt`	`BinaryOperator ... 'int' '+'`
中间代码	`test_unopt.ll`	`%10 = add nsw i32 %8, %9`
代码优化	`test_opt.ll`	直接传递常量 `6` 给 printf
目标代码	`test.s`	`callq printf`
调试支持	`test.pdb`	(Windows 特有的调试符号库)